

WEEK 4 · EXERCISE 16 · PUTTING IT TOGETHER

Capstone: a private task API

Put the pieces together: a to-do API where every user sees only their own tasks.

The big idea

Think of a hotel. Everyone uses the same front door and corridors (the **same URLs**), but your key card opens only **your** room.

In data terms: all tasks sit in one big table. For each request, filter it to the rows whose **owner** is **the logged-in user**, and only ever work with that.

Recap: how the pieces fit together

Model Task	The table: <code>owner</code> , <code>title</code> , <code>done</code> , <code>created_at</code> . Given to you.
Ownership	<code>owner</code> points to a user; <code>filter(owner=user)</code> gets their tasks.
Serializer	Task to JSON dict, and checks incoming data. Add a read-only <code>owner</code> .
ViewSet	One class for list, create, read, update and delete.
Router	Creates <code>/tasks/</code> and <code>/tasks/<id>/</code> and connects them to the viewset.
Permissions	<code>IsAuthenticated</code> : anonymous requests get 401.
New: <code>get_queryset</code>	Returns only the current user's tasks.
New: <code>perform_create</code>	Sets <code>owner</code> to the logged-in user on create.

One request's journey

Ada sends `GET /tasks/7/`, but task 7 belongs to Bob:

```
# 1. Router:      "/tasks/7/" matches the detail URL -> TaskViewSet
# 2. Permissions: is someone logged in? No -> 401. Yes (Ada) -> go on
# 3. get_queryset: take only Ada's tasks
# 4. Look up id 7 inside Ada's tasks: it isn't there -> 404
#    (if it were Ada's: the serializer turns it into JSON -> 200)
```

What the tests send

No single function this time: the tests send **HTTP requests** to your router's URLs.

```
# Users ada and bob, one task each: "Ada's task" and "Bob's task"
GET    /tasks/ (not logged in)          # 1: 401
# Now logged in as Ada:
GET    /tasks/                          # 2: 200, titles ["Ada's task"]
GET    /tasks/<Ada's id>/                # 3: 200, JSON below
POST   /tasks/ {"title": "New", "owner": <Bob's id>} # 4: 201, owner is Ada
GET    /tasks/<Bob's id>/                # 5: 404
PATCH or DELETE /tasks/<Bob's id>/      # 6: 404, Bob's task unchanged
PATCH then DELETE /tasks/<Ada's id>/    # 7: 200, then 204
POST   /tasks/ {}                       # 8: 400, a title is required

# test 3 expects:
{"id": 1, "title": "Ada's task", "done": False, "owner": "ada"}
```

New words

authenticated

Logged in: `request.user` is a real user.

401 Unauthorized

"I don't know who you are. Log in first."

404 Not Found

"Nothing here." Also for others' tasks, so they stay hidden.

scope a queryset

Filter the rows down to the ones this user may see.

read-only field

Shown in responses, ignored in requests.

override a method

Write your own version of a method the class already has.

Tool: overriding (same name, yours wins)

```
class Shop:
    def opening_message(self):
        return "Welcome!"
    def open(self):
        print(self.opening_message())

class TeaShop(Shop):
    def opening_message(self):          # replaces Shop's version
        return "Welcome! The tea is ready."

TeaShop().open()
```

OUTPUT

```
Welcome! The tea is ready.
```

Tool: ReadOnlyField (shown, never set)

```
class EmailDemoSerializer(serializers.ModelSerializer):
    owner_email = serializers.ReadOnlyField(source="owner.email") # owner, then email

    class Meta:
        model = Task
        fields = ["title", "owner_email"]

print(EmailDemoSerializer(task).data) # task belongs to grace
incoming = EmailDemoSerializer(data={"title": "Plan", "owner_email": "x@evil.io"})
print(incoming.is_valid(), incoming.validated_data)
```

OUTPUT

```
{'title': 'Write report', 'owner_email': 'grace@example.com'}
True {'title': 'Plan'}
```

Tool: `serializer.save(owner=...)`

Keywords you pass to `save()` are **added** to the checked data: the server fills them in.

```
class TitleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = ["id", "title"]

incoming = TitleSerializer(data={"title": "Plan the trip"})
incoming.is_valid()
task = incoming.save(owner=grace)      # grace is a User
print(task.owner.username, task.title)
```

OUTPUT

```
grace Plan the trip
```

queryset = ... vs get_queryset(self)

queryset = Task.objects.all()

- A class variable, fixed when the file loads
- Can't know who is asking
- Everyone sees everyone's tasks
- The router guesses the URL name from it

def get_queryset(self):

- A method, called **for each request**
- Can use `self.request.user`
- Returns only this user's tasks
- Needs `basename=` on the router

Tool: `.filter(...)` and `.order_by(...)`

Keep the matching rows, then sort them, like `df.sort_values("title")` in pandas.

```
Task.objects.create(owner=grace, title="B task", done=True)
Task.objects.create(owner=grace, title="A task", done=True)
Task.objects.create(owner=grace, title="C task")

finished = Task.objects.filter(done=True).order_by("title")
print([t.title for t in finished])
```

OUTPUT

```
['A task', 'B task']
```

Your task, step by step

- 1 Serializer: a `ReadOnlyField` owner whose `source` ends in `.username`
- 2 Add `"owner"` to `Meta.fields` → helps test 3
- 3 Viewset: `permission_classes = [IsAuthenticated]` → helps test 1
- 4 Replace `queryset` with `get_queryset`: my tasks, ordered by `"id"` → helps 2, 5, 6
- 5 Router: register the viewset at `"tasks"` with `basename="task"` → 1-3, 5-8
- 6 `perform_create(self, serializer)`: save with `owner=self.request.user` → test 4

Common mistakes

- Removing `queryset` but forgetting `basename="task": AssertionError`
- Keeping `Task.objects.all()`: everyone sees everything
- `request.user` inside a method: it's `self.request.user`
- `source="owner"` without `.username`: test 3 wants "ada"
- Taking the owner from `request.data`: never trust the client

Where you'll use this

Almost every real DRF endpoint that holds user data has this shape: notes, orders, messages, invoices. Log in, scope the queryset to the user, set the owner on the server.

- "Broken access control" is the most common serious web security bug
- Scoping `get_queryset()` is the simplest, strongest defence
- Always test the boundary: can user A reach user B's data?

Check yourself

1. Ada asks for Bob's task. Why is 404 a good answer?
2. Why can't `queryset = ...` filter by the current user?
3. A client sends `"owner": 2` when creating a task. What happens, and why?
4. Which piece turns anonymous requests away, and with which status?

Keep learning

[Use Django REST Framework to Create Web APIs \(video course\)](#)

freeCodeCamp · video · beginner. Start here: a free two-hour video that builds a full DRF API with models, serializers, viewsets, auth and tests.

[Tutorial 4: Authentication & Permissions](#)

Django REST framework · tutorial · beginner. Shows perform_create saving owner=self.request.user, the same trick your TaskViewSet uses.

[Filtering against the current user](#)

Django REST framework · docs · intermediate. The official example of overriding get_queryset so each user only ever sees their own rows.

[Generic views](#)

Django REST framework · docs · intermediate. Reference for get_queryset, perform_create and the other hooks a ModelViewSet inherits.

[A01 Broken Access Control \(OWASP Top 10:2025\)](#)

OWASP · article · intermediate. Why letting one user read or edit another user's task is the most common serious API bug.